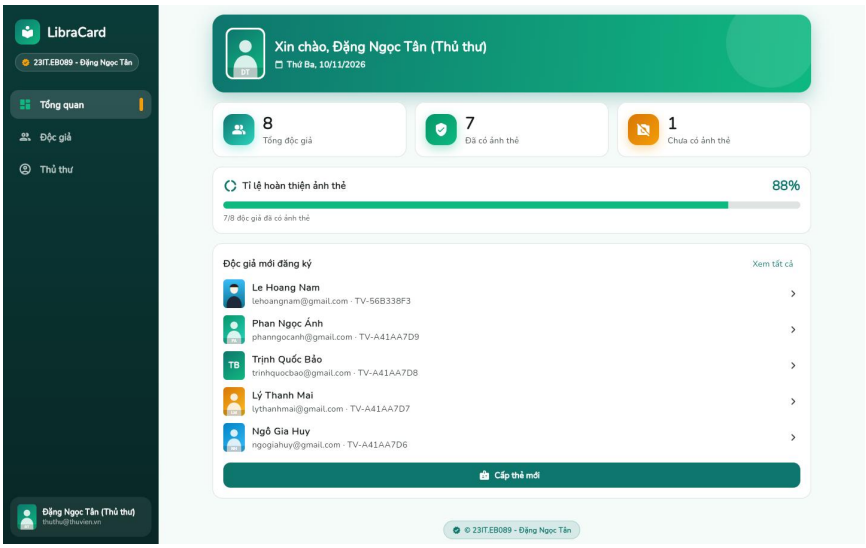
# BÁO CÁO ĐỀ THI GIỮA KỲ

## HỌC PHẦN: LẬP TRÌNH ĐA NỀN TẢNG

ĐỀ TÀI

# LIBRACARD

*Ứng dụng quản lý độc giả và cấp thẻ thư viện đa nền tảng (Web & Android)*




| Thông tin | Nội dung |
|---|---|
| Sinh viên thực hiện | Đặng Ngọc Tân |
| Mã số sinh viên | 23IT.EB089 |
| Công nghệ | Flutter 3.47 · Node.js/Express · MongoDB 8 |
| Nền tảng chạy thử | Web (Chrome) · Android 14 (Pixel 8 emulator) |
| Link GitHub | https://github.com/ngoctan2503/GiuaKiDNT-23IT.EB089 |
| Link video demo | (dán link video demo tại đây) |

# MỤC LỤC

1. Giới thiệu đề tài ..... 3
    1.1. Đối chiếu yêu cầu của đề ..... 3
    1.2. Công nghệ sử dụng ..... 3
    1.3. Kiến trúc hệ thống ..... 3
2. Thiết kế cơ sở dữ liệu ..... 4
3. Các bước thực hiện ..... 5
    3.1. Chuẩn bị môi trường ..... 5
    3.2. Xây dựng REST API (thư mục server) ..... 6
    3.3. Xây dựng ứng dụng Flutter (thư mục app) ..... 7
4. Kết quả chạy trên nền tảng Web ..... 8
5. Kết quả chạy trên nền tảng di động (Android) ..... 16
6. Các chức năng nâng cao ..... 20
7. Hướng dẫn chạy và sản phẩm nộp ..... 21
8. Kết luận ..... 21

*(Trong Word: nhấn chuột phải vào mục lục và chọn "Update Field" để cập nhật số trang.)*

# 1. Giới thiệu đề tài

Thư viện cần quản lý danh sách độc giả và cấp cho mỗi người một thẻ thư viện có ảnh. Đề tài "LibraCard" xây dựng phân hệ dành cho thủ thư (admin): cấp thẻ cho độc giả mới (thêm), xem danh sách và thẻ thư viện điện tử (hiển thị), cập nhật thông tin (sửa) và thu hồi thẻ (xóa).

Ứng dụng viết bằng Flutter nên một mã nguồn chạy được cả trên Web và Android. Dữ liệu lưu trong MongoDB thông qua một REST API viết bằng Node.js/Express.

Theo yêu cầu của đề, mỗi độc giả trong CSDL chỉ có 4 trường: email, matkhau, hoten, hinhanh (ảnh thẻ). Ứng dụng không thêm trường nào khác. "Mã thẻ" hiển thị trên thẻ được tính từ khóa _id do MongoDB tự sinh, không lưu thành trường riêng. Thủ thư đăng nhập bằng chính một tài khoản trong collection này.

## 1.1. Đối chiếu yêu cầu của đề

| Câu | Yêu cầu | Cách thực hiện trong ứng dụng |
|---|---|---|
| 1 | Dùng CSDL (Firebase/MongoDB) thêm và hiển thị email, matkhau, hoten, hinhanh; chạy trên Web (8.0đ) | MongoDB + REST API. Trên Web: cấp thẻ mới (thêm độc giả kèm ảnh), bảng danh sách độc giả, thẻ thư viện chi tiết, trang tổng quan. |
| 2 | Đăng nhập, xóa, sửa và các chức năng nâng cao (1.5đ) | Đăng nhập thủ thư (JWT), cập nhật, thu hồi thẻ có xác nhận, tìm kiếm realtime, lọc theo ảnh thẻ, sắp xếp tên kiểu tiếng Việt, thống kê, xem trước thẻ, mã vạch, đổi mật khẩu, dark mode, bcrypt. |
| 3 | Nhập hình ảnh và hiển thị thông tin người dùng trên di động (0.5đ) | Trên Android chọn ảnh thẻ từ thư viện hoặc chụp bằng camera (image_picker); ảnh hiển thị ở danh sách và trên thẻ thư viện. |

## 1.2. Công nghệ sử dụng

- **Flutter 3.47 / Dart 3.13:** xây dựng giao diện cho Web và Android từ một mã nguồn.
- **Node.js 24 + Express 5:** REST API trung gian giữa app và CSDL.
- **MongoDB 8.0 + Mongoose:** lưu trữ collection docgia.
- **Thư viện Flutter:** http, image_picker, shared_preferences, google_fonts, flutter_animate, flutter_svg, intl.
- **Thư viện Node.js:** express, mongoose, bcryptjs, jsonwebtoken, cors, dotenv, nodemon.

## 1.3. Kiến trúc hệ thống




Trên Web app gọi API qua địa chỉ http://localhost:3000. Trên Android emulator, app gọi qua địa chỉ http://10.0.2.2:3000, là địa chỉ máy thật nhìn từ trong máy ảo.

## 2. Thiết kế cơ sở dữ liệu

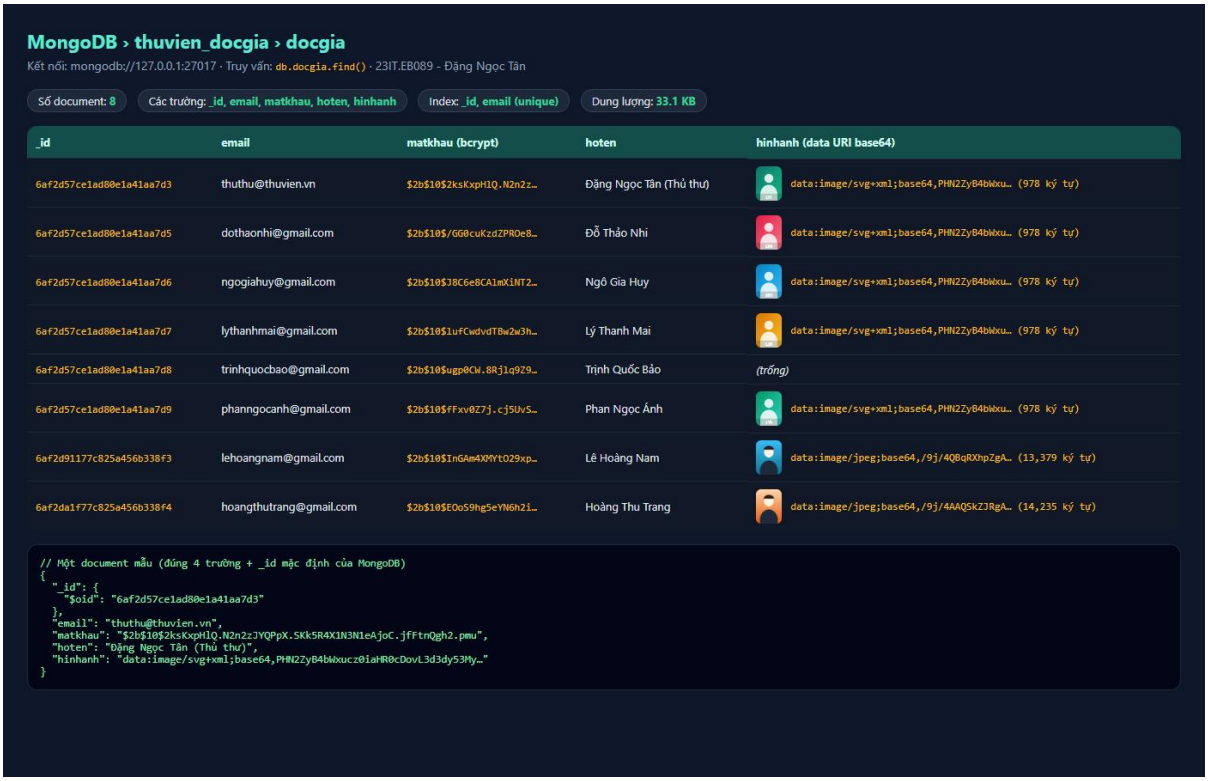
Cơ sở dữ liệu tên thuvien_docgia, có một collection docgia. Mỗi document là một độc giả (kể cả tài khoản thủ thư) và có đúng 4 trường theo yêu cầu:

| Trường | Kiểu | Mô tả |
|---|---|---|
| email | String | Email đăng nhập của độc giả, duy nhất (unique index), tự chuyển về chữ thường. |
| matkhau | String | Mật khẩu đã mã hóa bằng bcrypt (không lưu dạng thô). |
| hoten | String | Họ và tên độc giả (bắt buộc). |
| hinhanh | String | Ảnh thẻ 3×4 lưu dạng chuỗi data URI base64 (ảnh được nén tối đa 600×800, chất lượng 70%). |

Lưu ảnh thẻ dưới dạng chuỗi base64 ngay trong trường hinhanh giúp file CSDL xuất ra (docgia.json) chứa đầy đủ hình ảnh, không phụ thuộc vào dịch vụ lưu trữ file bên ngoài.

**Định nghĩa model (server/src/models/DocGia.js):** tắt versionKey để MongoDB không sinh thêm trường __v và không dùng timestamps.

```
const mongoose = require('mongoose');
const schema = new mongoose.Schema(
  {
    email: { type: String, required: true, unique: true, lowercase: true, trim: true },
    matkhau: { type: String, required: true },
    hoten: { type: String, required: true, trim: true },
    hinhanh: { type: String, default: '' },
  },
  {
    versionKey: false,
    collection: 'docgia',
    toJSON: {
      transform: (doc, ret) => {
        delete ret.matkhau;
        return ret;
      },
    },
  }
);
module.exports = mongoose.model('DocGia', schema);
```




*Hình 1. Dữ liệu thực tế trong collection docgia (truy vấn db.docgia.find()), chỉ gồm _id, email, matkhau, hoten, hinhanh*

**File CSDL nộp kèm:** database/docgia.json, xuất bằng lệnh npm run export theo định dạng mongoexport --jsonArray. Có thể nạp lại vào MongoDB bằng lệnh npm run import. Trích một document:

```
{
  "_id": { "$oid": "6af2d57ce1ad80e1a41aa7d3" },
  "email": "thuthu@thuvien.vn",
  "matkhau": "$2b$10$2ksKxpHlQ.N2n2zJYQPpX.SKk5R4X1N3N1eAjoC.jfFtnQgh2.pmu",
  "hoten": "Đặng Ngọc Tân (Thủ thư)",
  "hinhanh": "..."
}
```

**Mã thẻ không phải là trường:** mã thẻ "TV-XXXXXXXX" trên giao diện được tính từ 8 ký tự cuối của _id (getter maThe trong lớp DocGia của Flutter). Nhờ vậy CSDL vẫn chỉ có đúng 4 trường theo yêu cầu.

# 3. Các bước thực hiện

## 3.1. Chuẩn bị môi trường

**Bước 1.** Cài Flutter SDK, Node.js, Android Studio và tạo máy ảo Pixel 8 (Android 14). Kiểm tra bằng các lệnh:

```
flutter --version        # Flutter 3.47.9, Dart 3.13.7
node --version           # v24.21.0
flutter devices          # emulator-5554, chrome, edge
```

**Bước 2.** Tải MongoDB Community Server 8.0.4 bản zip (không cần cài đặt), giải nén vào D:\dev\mongodb rồi khởi động:

```
mongod --dbpath server/data/db --port 27017 --bind_ip 127.0.0.1
# hoặc chạy file start-mongodb.bat trong thư mục dự án
```

## 3.2. Xây dựng REST API (thư mục server)

**Bước 3.** Khởi tạo dự án và cài thư viện:

```
mkdir server && cd server
npm init -y
npm i express mongoose bcryptjs jsonwebtoken cors dotenv
npm i -D nodemon
```

**Bước 4.** Tạo cấu trúc mã nguồn:

```
server/
 ├─ src/index.js            # khởi tạo Express, CORS, routes
 ├─ src/db.js, config.js    # kết nối MongoDB, biến môi trường
 ├─ src/models/DocGia.js    # schema 4 trường
 ├─ src/middleware/auth.js  # xác thực JWT
 ├─ src/routes/auth.js      # đăng nhập thủ thư, đổi mật khẩu
 ├─ src/routes/docgia.js    # CRUD độc giả, thống kê
 └─ scripts/seed.js, export.js, import.js
```

**Bước 5.** Xây dựng API đăng nhập: tìm tài khoản theo email, so sánh mật khẩu bằng bcrypt và trả về JWT.

```
router.post('/login', async (req, res, next) => {
  const { email, matkhau } = req.body || {};
  const user = await DocGia.findOne({ email: String(email).toLowerCase().trim() });
  if (!user || !(await bcrypt.compare(String(matkhau), user.matkhau)))
    return res.status(401).json({ message: 'Email hoặc mật khẩu không đúng' });
  const token = jwt.sign({ id: user._id.toString() }, JWT_SECRET, { expiresIn: '7d' });
  res.json({ token, user });
});
```

**Bước 6.** Xây dựng các API quản lý độc giả (đều yêu cầu header Authorization: Bearer <token>):

| Method | Endpoint | Chức năng |
|---|---|---|
| POST | /api/auth/login | Thủ thư đăng nhập, trả về token và thông tin |
| PUT | /api/auth/change-password | Đổi mật khẩu của thủ thư đang đăng nhập |
| GET | /api/docgia?q=&sort= | Danh sách, tìm kiếm theo email/họ tên, sắp xếp |
| GET | /api/docgia/stats | Thống kê tổng số, đã có ảnh thẻ, chưa có ảnh |
| GET | /api/docgia/:id | Xem chi tiết độc giả |
| POST | /api/docgia | Cấp thẻ mới (mã hóa mật khẩu, kiểm tra email trùng → 409) |
| PUT | /api/docgia/:id | Cập nhật độc giả (bỏ trống mật khẩu = giữ nguyên) |
| DELETE | /api/docgia/:id | Thu hồi thẻ (không cho xóa chính mình) |

**Bước 7.** Nạp dữ liệu mẫu và chạy server:

```
npm run seed     # tạo thuthu@thuvien.vn / thuthu123 và 6 độc giả mẫu
npm start        # API chạy tại http://localhost:3000
npm run export   # xuất CSDL ra database/docgia.json
```

## 3.3. Xây dựng ứng dụng Flutter (thư mục app)

**Bước 8.** Tạo dự án cho 2 nền tảng Web và Android, rồi cài thư viện:

```
flutter create app --platforms=web,android --org com.dangngoctan --project-name libracard
cd app
flutter pub add http image_picker shared_preferences google_fonts flutter_animate intl
flutter_svg
```

**Bước 9.** Tổ chức mã nguồn:

```
lib/
 ├─ main.dart, config.dart, theme.dart, utils.dart
 ├─ models/doc_gia.dart             # email, hoten, hinhanh (+ _id, maThe tính từ _id)
 ├─ services/api_service.dart       # gọi REST API bằng http
 ├─ services/auth_service.dart      # lưu token (shared_preferences), dark mode
 ├─ screens/login_screen.dart, home_shell.dart, dashboard_screen.dart,
 │          reader_list_screen.dart, reader_form_screen.dart,
 │          reader_card_screen.dart, librarian_screen.dart
 └─ widgets/library_card.dart (thẻ + mã vạch), reader_photo.dart, ...
```

**Bước 10.** Cấu hình địa chỉ API theo nền tảng (lib/config.dart):

```
import 'package:flutter/foundation.dart';

class AppConfig {
  static String get baseUrl =>
      kIsWeb ? 'http://localhost:3000' : 'http://10.0.2.2:3000';
  static const String appName = 'LibraCard';
  static const String author = '23IT.EB089 - Đặng Ngọc Tân';
}
```

**Bước 11.** Cấp quyền cho Android trong AndroidManifest.xml: INTERNET, CAMERA và android:usesCleartextTraffic="true" để gọi API http nội bộ.

**Bước 12.** Chọn ảnh thẻ và chuyển thành chuỗi base64 để lưu vào trường hinhanh. Đoạn code này dùng chung cho Web và Android:

```
final f = await ImagePicker().pickImage(
    source: source,            // ImageSource.gallery hoặc ImageSource.camera
    maxWidth: 600, maxHeight: 800, imageQuality: 70);
if (f == null) return;
final bytes = await f.readAsBytes();
setState(() => _hinhanh = 'data:${_mimeOf(f)}; base64,${base64Encode(bytes)}');
```

**Bước 13.** Vẽ thẻ thư viện: widget LibraryCard hiển thị ảnh thẻ, họ tên, email, mã thẻ, và một dải mã vạch do CustomPainter vẽ từ các ký tự hex của _id.

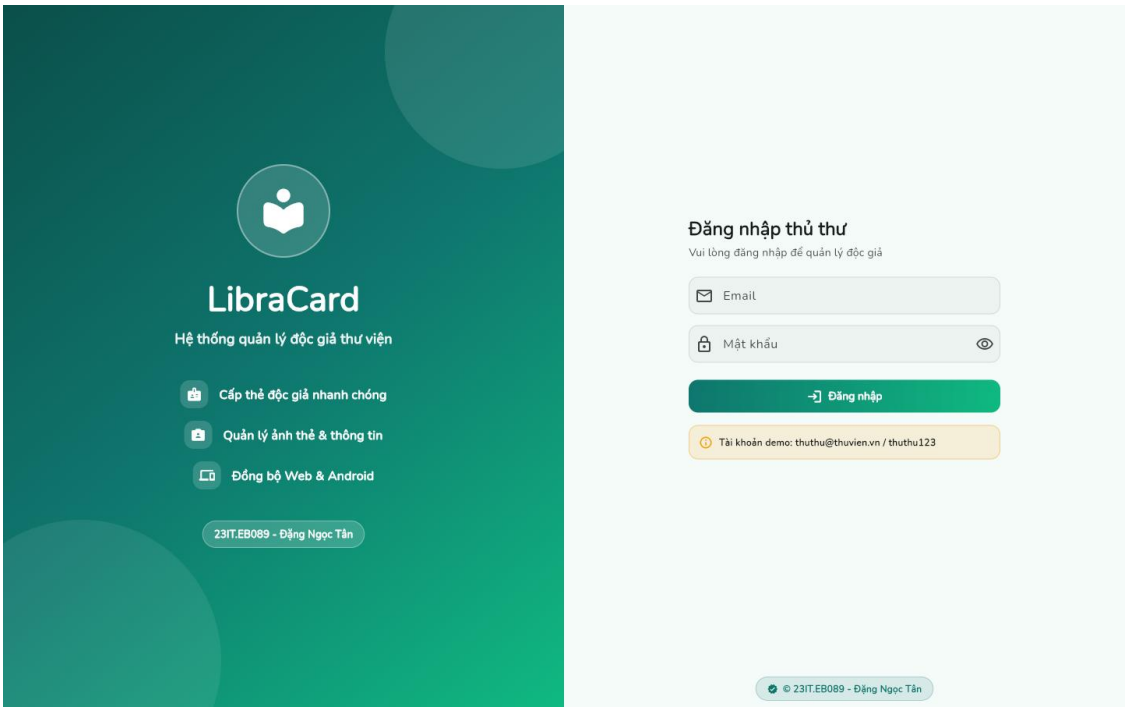
**Bước 14.** Chạy ứng dụng:

```
flutter run -d chrome          # nền tảng Web
flutter run -d emulator-5554   # Android emulator Pixel 8
flutter build web --release    # build bản web
```

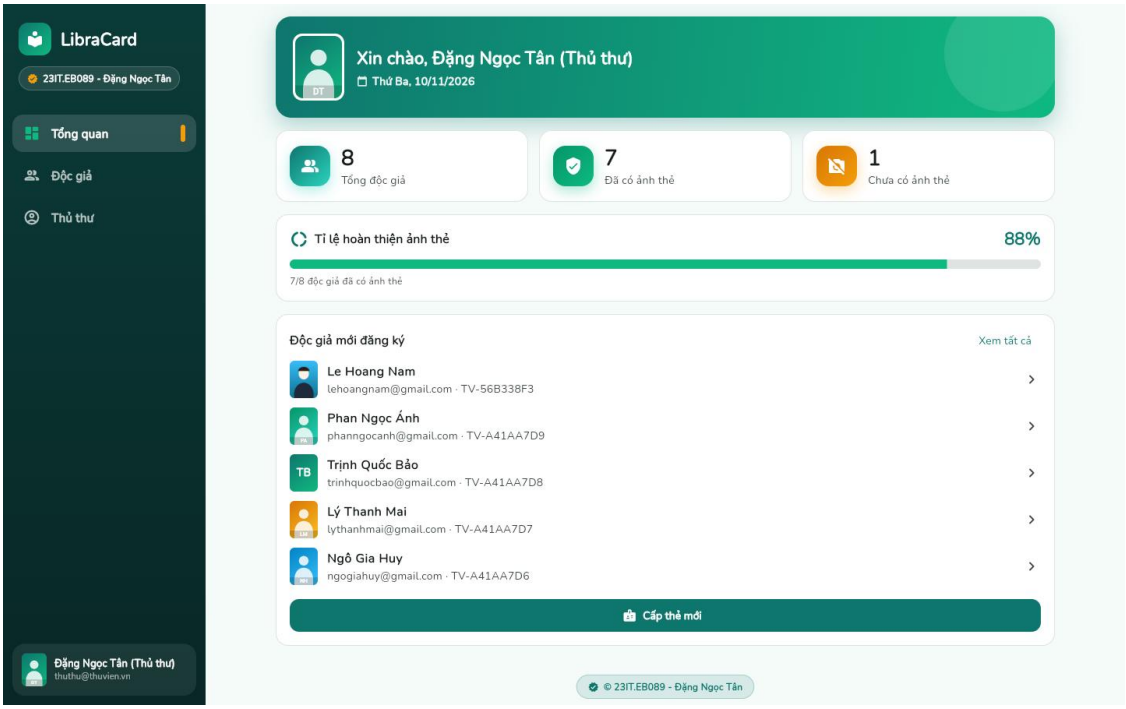
**Lưu ý khi build Android:** vì Pub cache (ổ C:) và dự án (ổ D:) nằm khác ổ đĩa, Kotlin incremental báo lỗi "different roots". Khắc phục bằng cách thêm dòng kotlin.incremental=false vào android/gradle.properties.

## 4. Kết quả chạy trên nền tảng Web

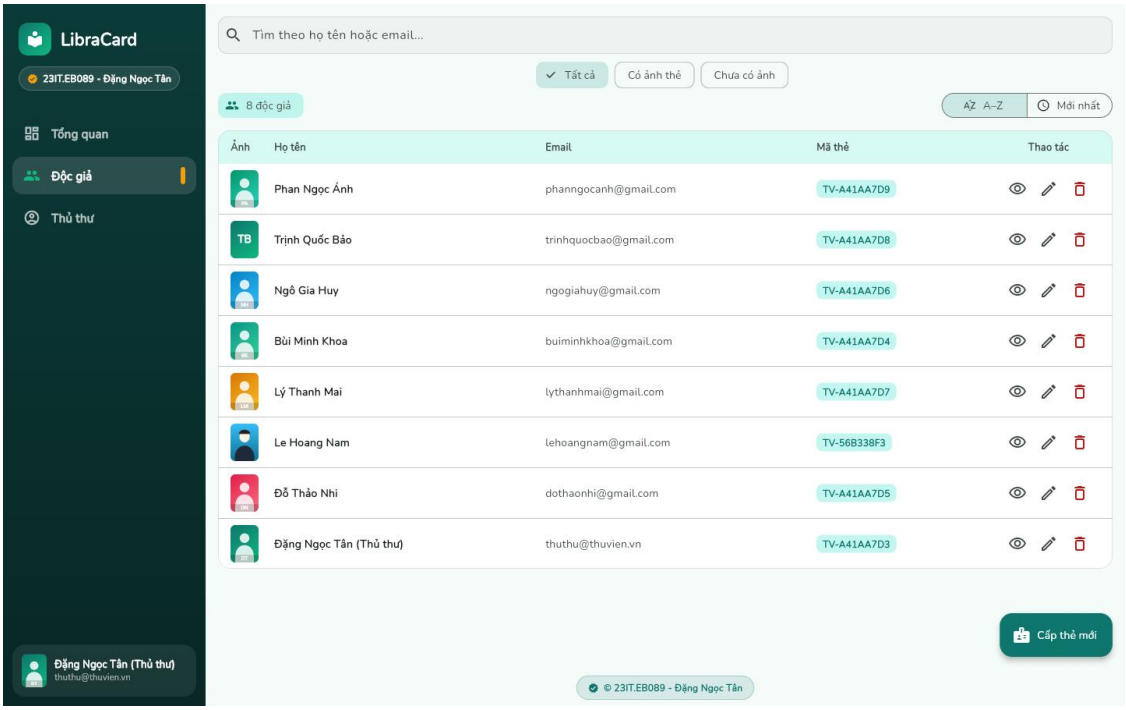
Các ảnh dưới đây được chụp từ bản build web (flutter build web) chạy trên trình duyệt Chrome ở độ phân giải 1440×900. Trên màn hình rộng, giao diện dùng thanh menu bên trái và danh sách dạng bảng.



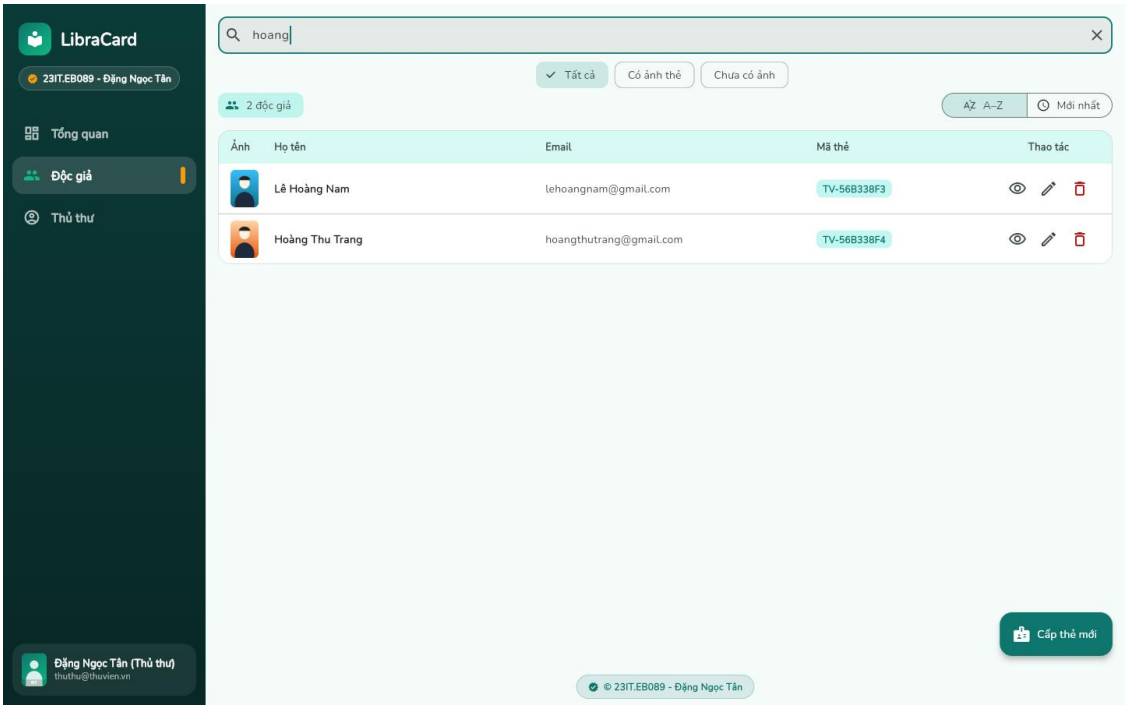

*Hình 2. Màn hình đăng nhập thủ thư (Web) dạng chia đôi màn hình*



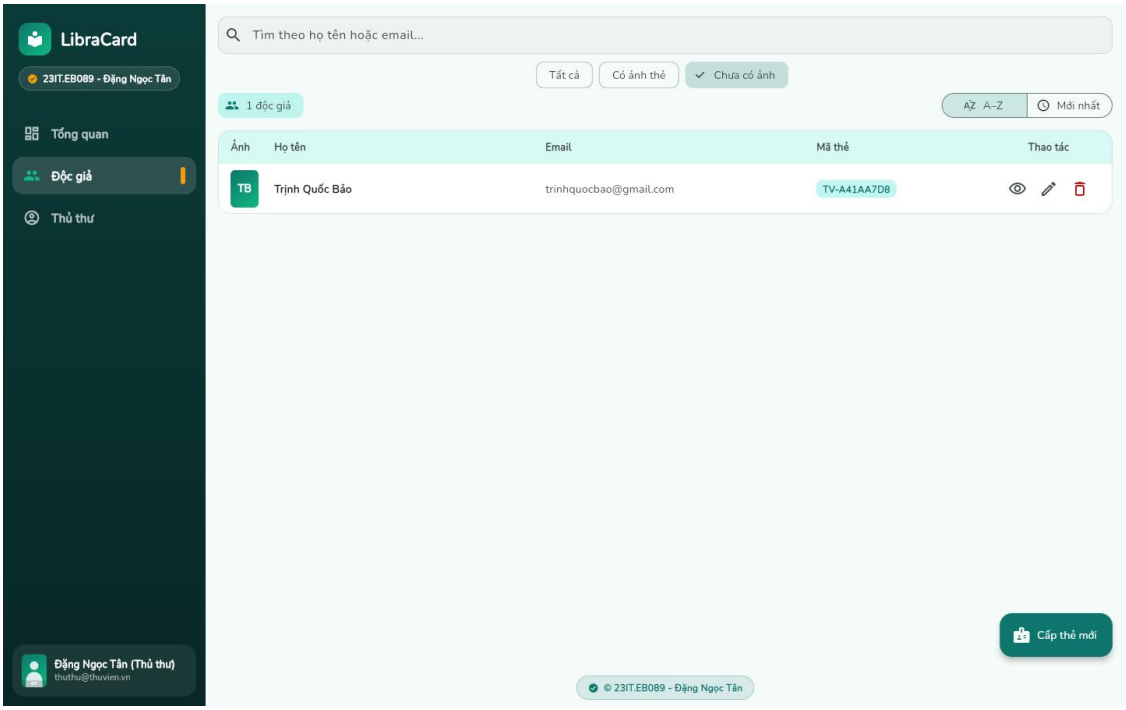

*Hình 3. Trang Tổng quan: thống kê, tỉ lệ hoàn thiện ảnh thẻ, độc giả mới đăng ký*



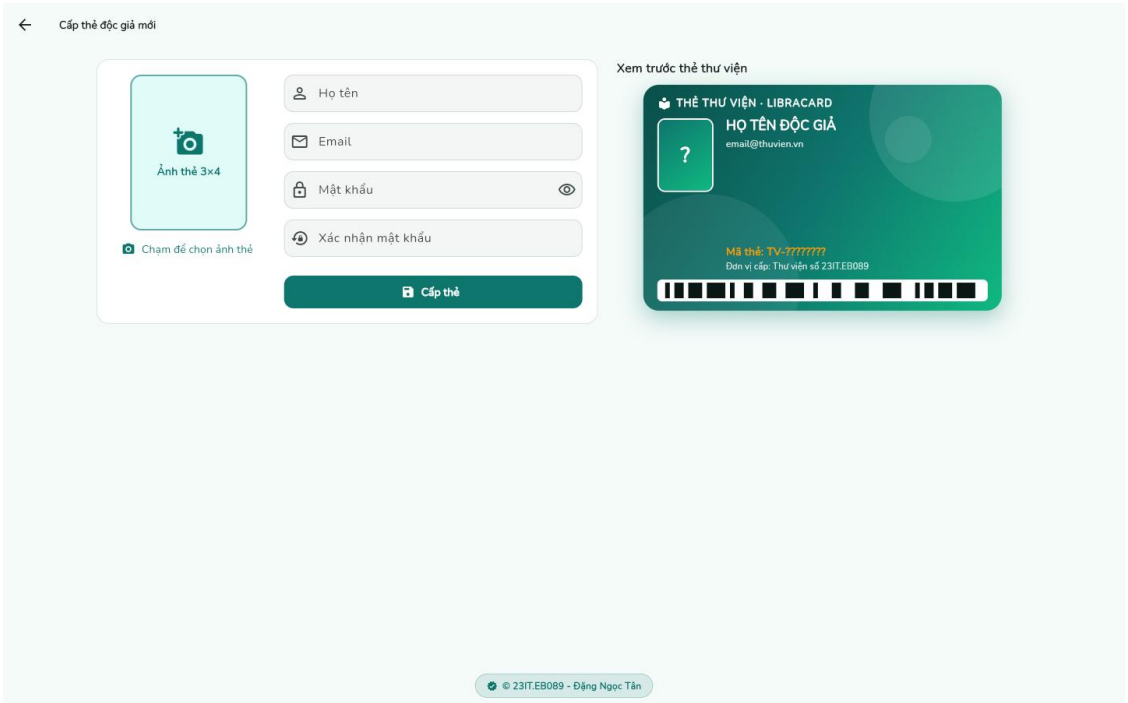

*Hình 4. Bảng danh sách độc giả: ảnh thẻ, họ tên, email, mã thẻ, thao tác*



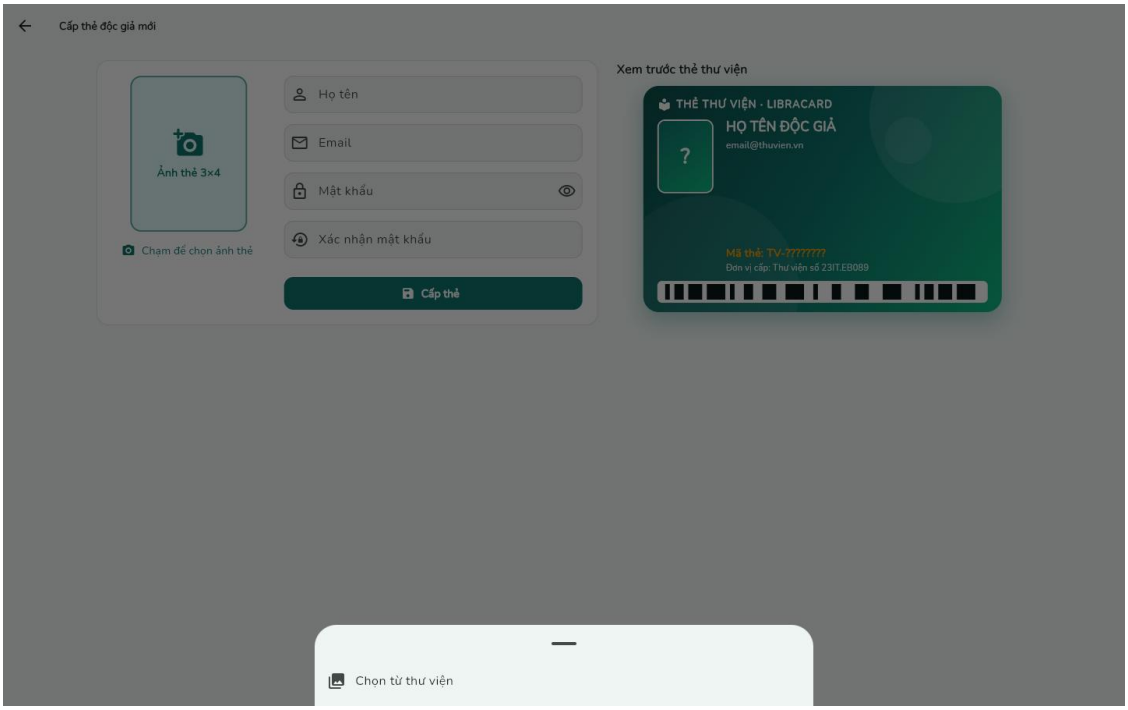

*Hình 5. Tìm kiếm realtime theo họ tên/email ("hoang")*



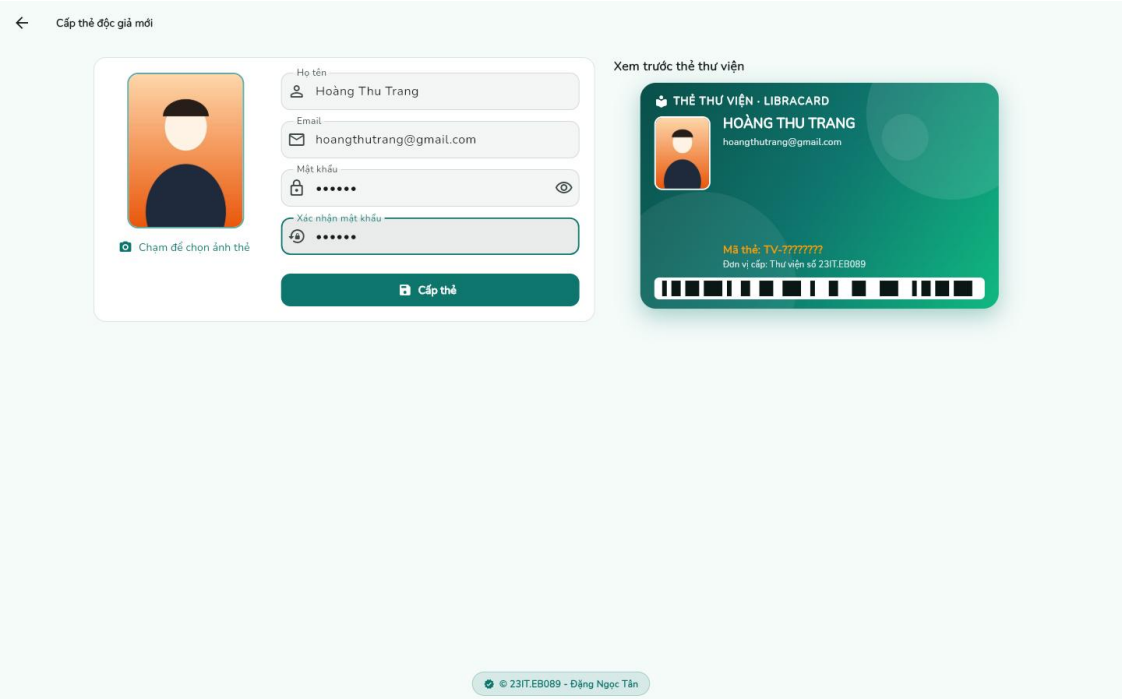

*Hình 6. Lọc độc giả "Chưa có ảnh" thẻ*



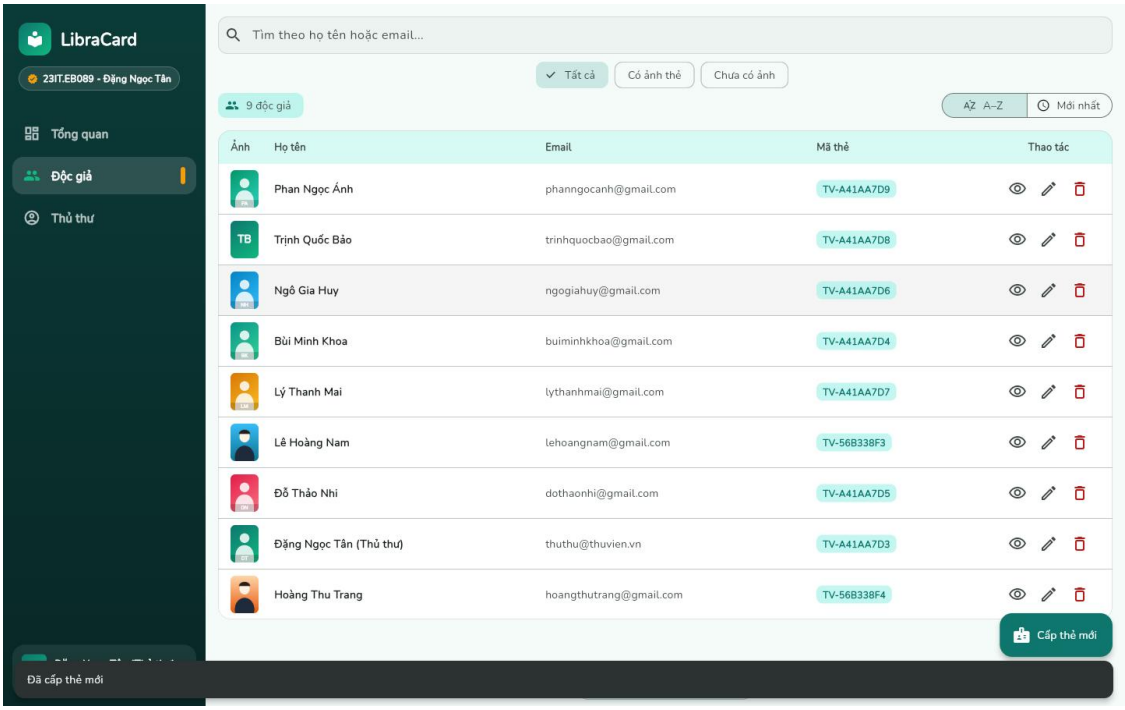

*Hình 7. Form cấp thẻ độc giả mới, kèm xem trước thẻ thư viện*



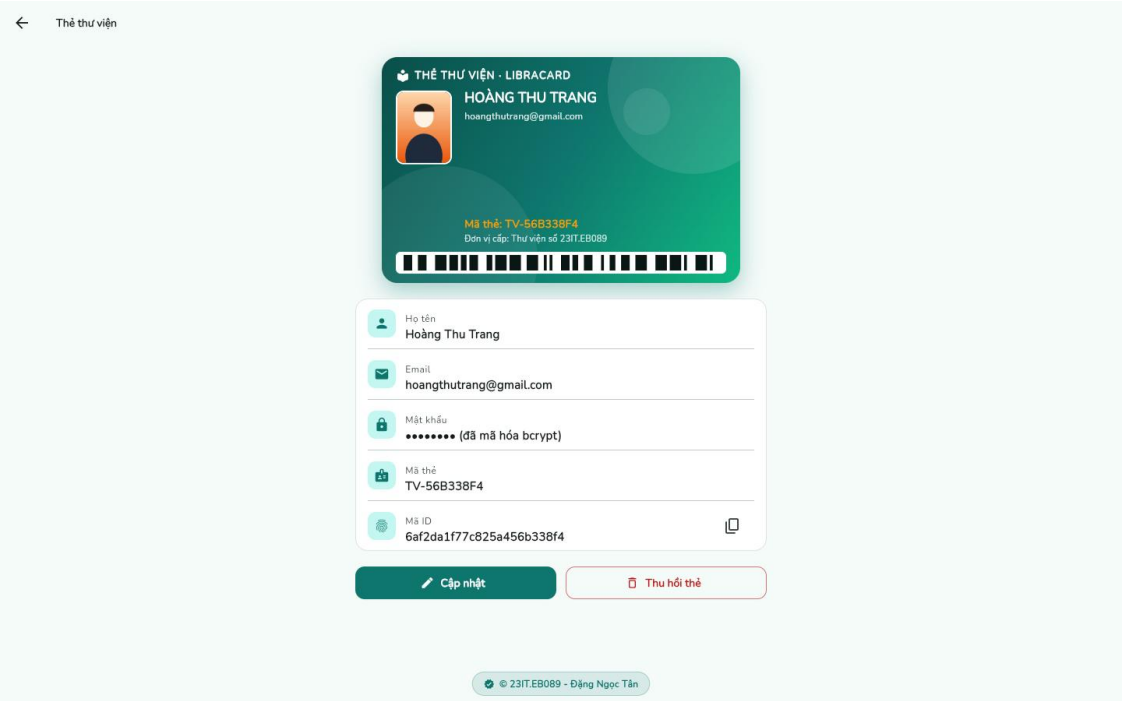

*Hình 8. Chọn ảnh thẻ từ máy tính*



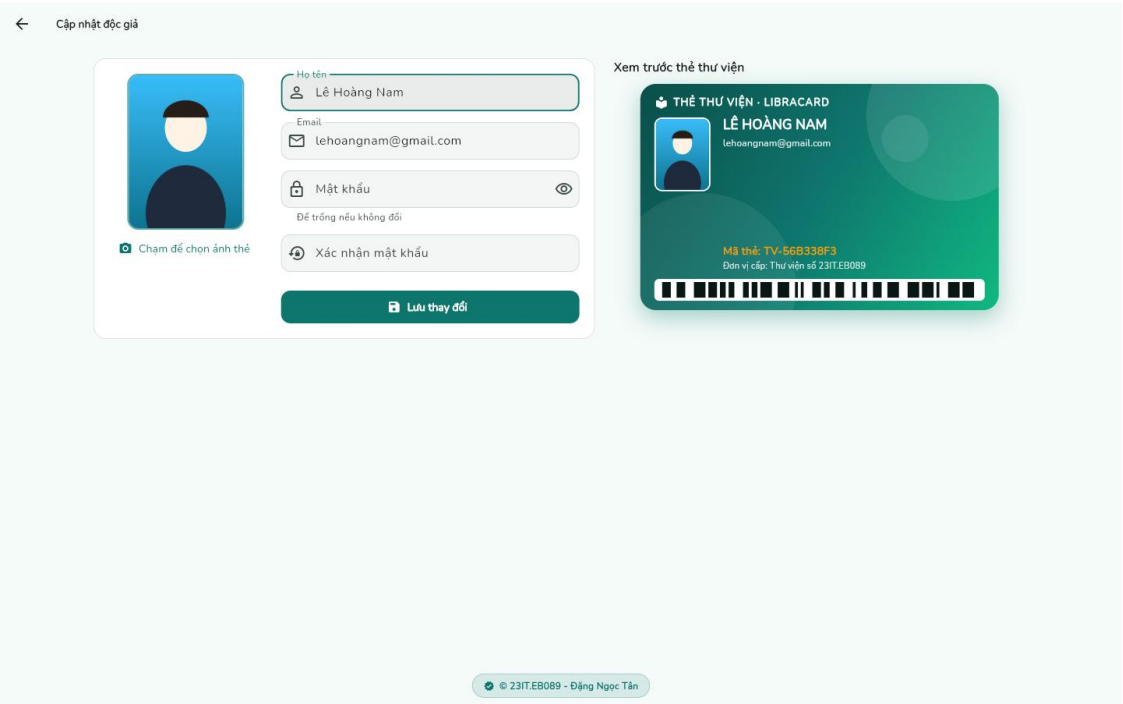

*Hình 9. Sau khi chọn ảnh và nhập thông tin, thẻ xem trước cập nhật ngay*



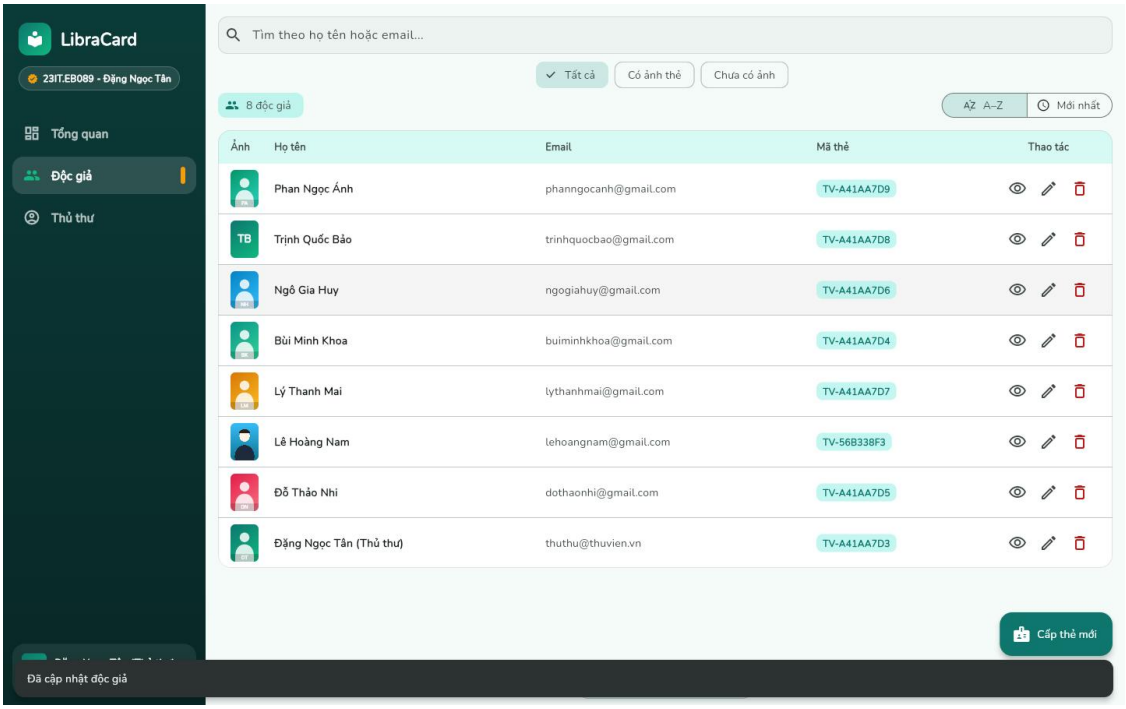

*Hình 10. Cấp thẻ thành công, "Hoàng Thu Trang" xuất hiện trong danh sách*



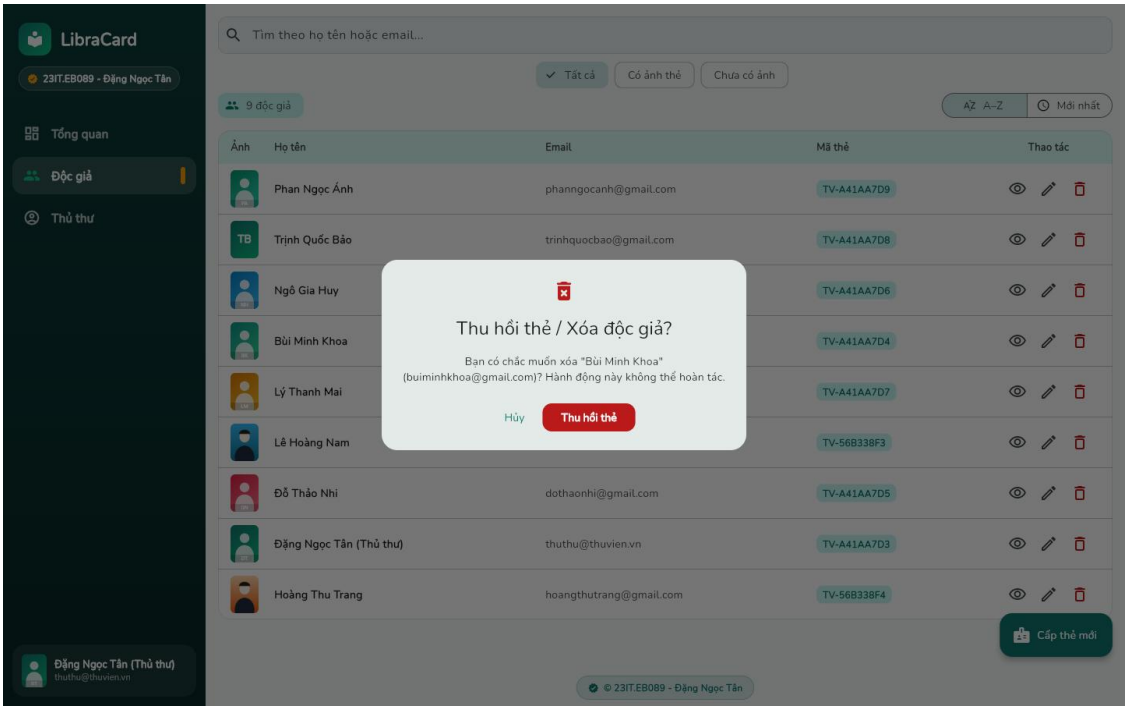

*Hình 11. Thẻ thư viện điện tử của độc giả: ảnh thẻ, mã thẻ, mã vạch, thông tin chi tiết*



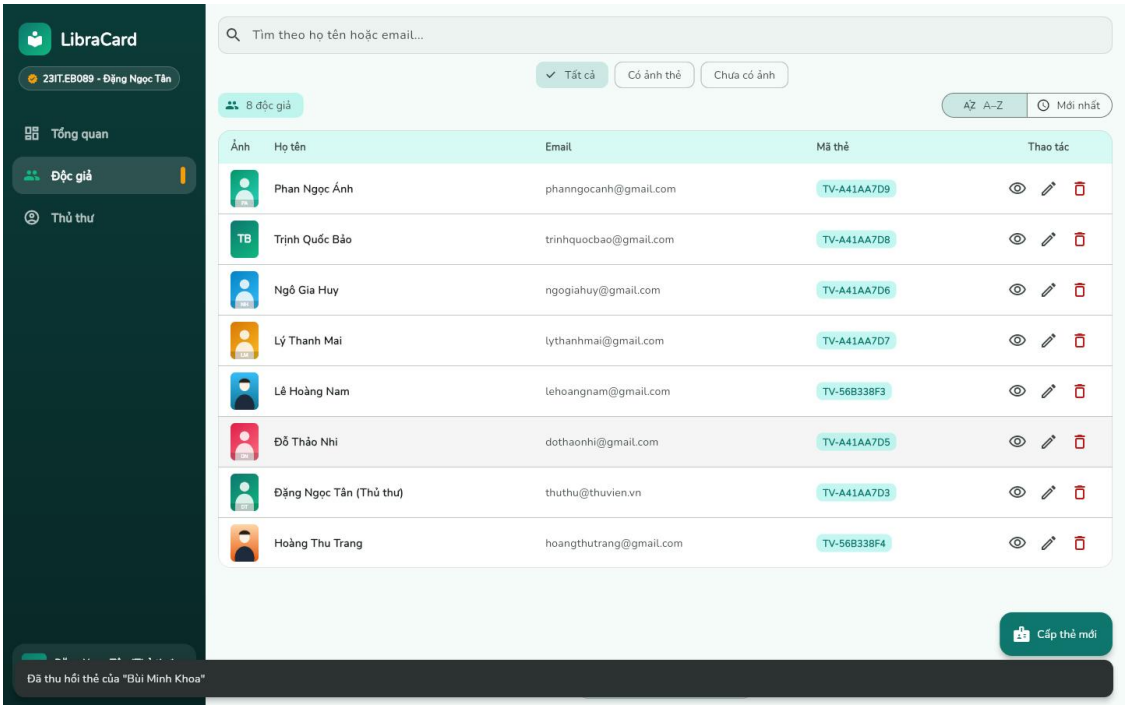

*Hình 12. Cập nhật độc giả: sửa họ tên thành "Lê Hoàng Nam" (mật khẩu để trống thì giữ nguyên)*



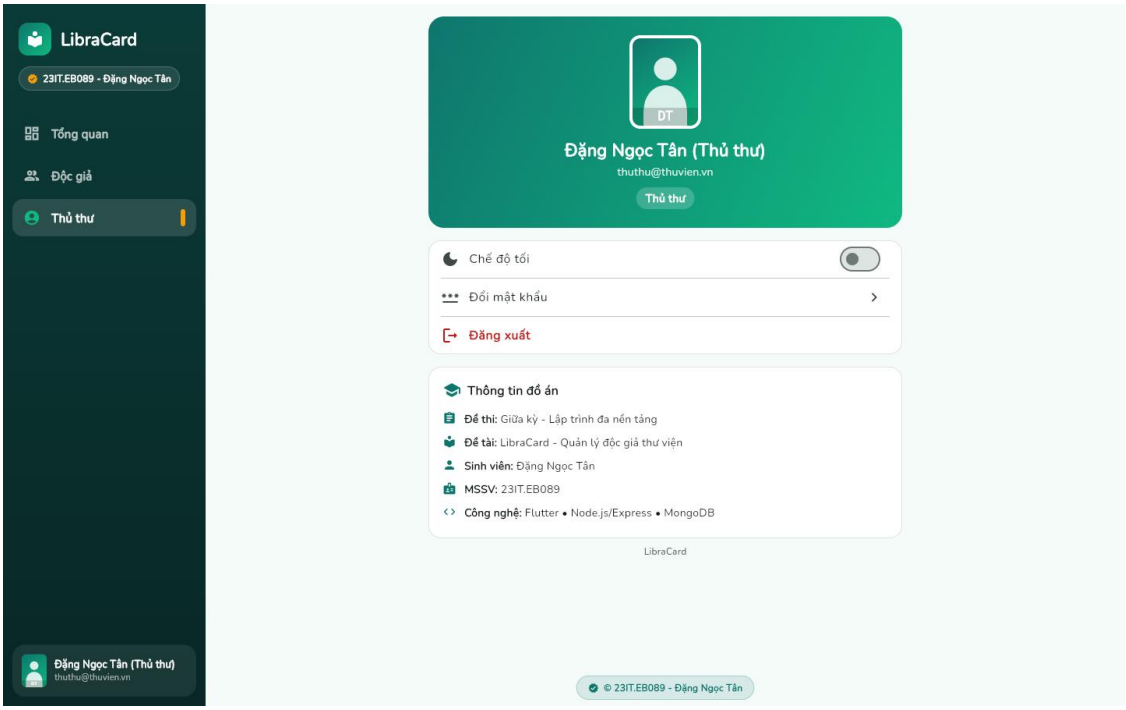

*Hình 13. Cập nhật thành công*



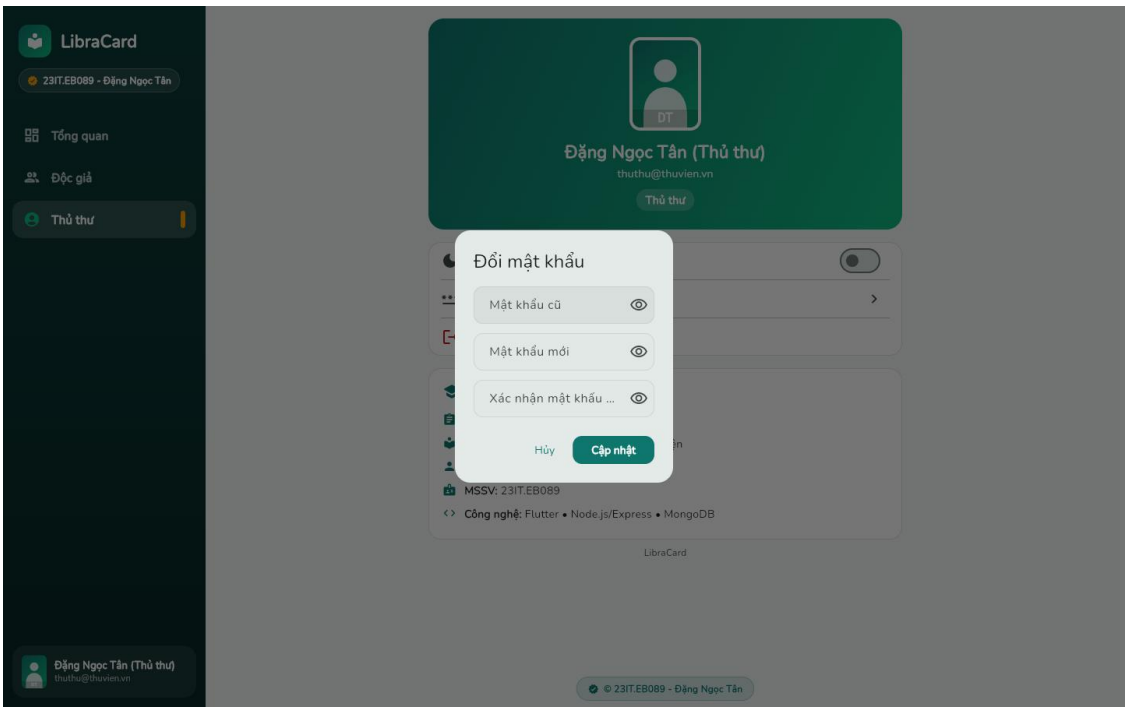

*Hình 14. Hộp thoại xác nhận thu hồi thẻ / xóa độc giả*



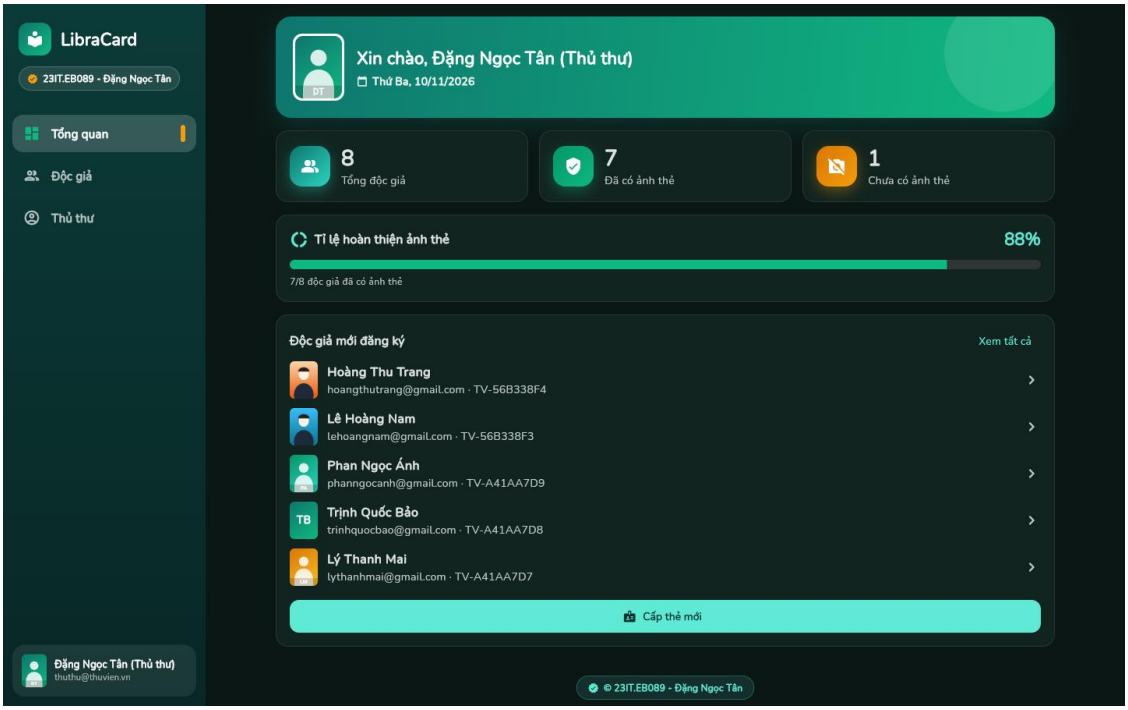

*Hình 15. Thu hồi thẻ thành công, danh sách còn 8 độc giả*




*Hình 16. Trang Thủ thư: dark mode, đổi mật khẩu, đăng xuất, thông tin đồ án*



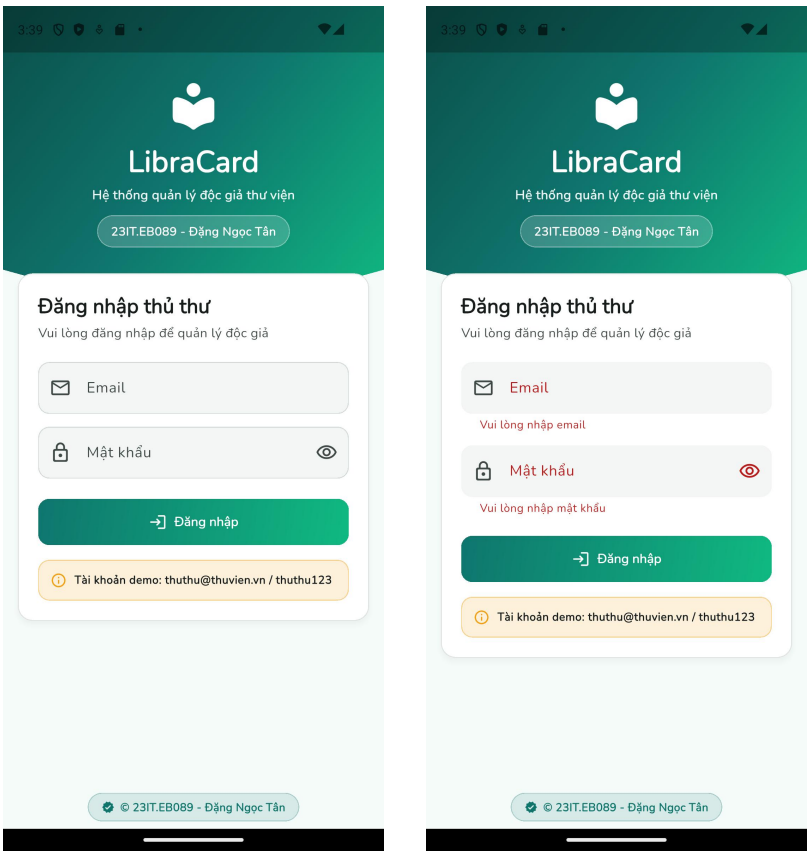

*Hình 17. Hộp thoại đổi mật khẩu*



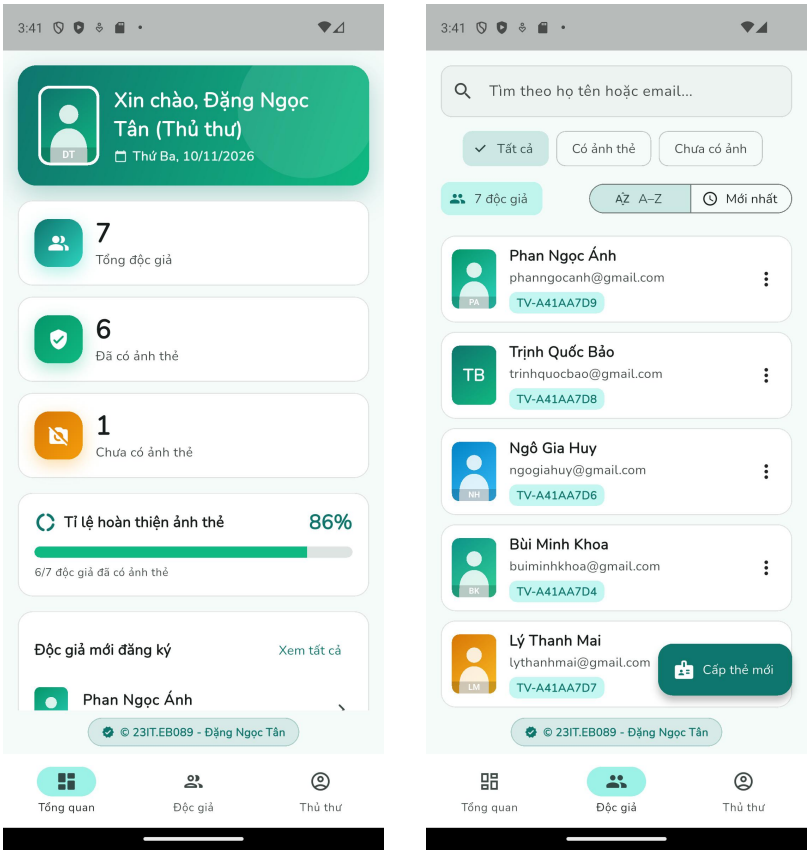

*Hình 18. Giao diện chế độ tối (Dark mode)*

## 5. Kết quả chạy trên nền tảng di động (Android)

Ứng dụng chạy trên máy ảo Pixel 8 (Android 14, API 34) của Android Studio, dùng chung mã nguồn với bản Web. Trên màn hình hẹp, giao diện dùng thanh điều hướng ở dưới và danh sách dạng thẻ.

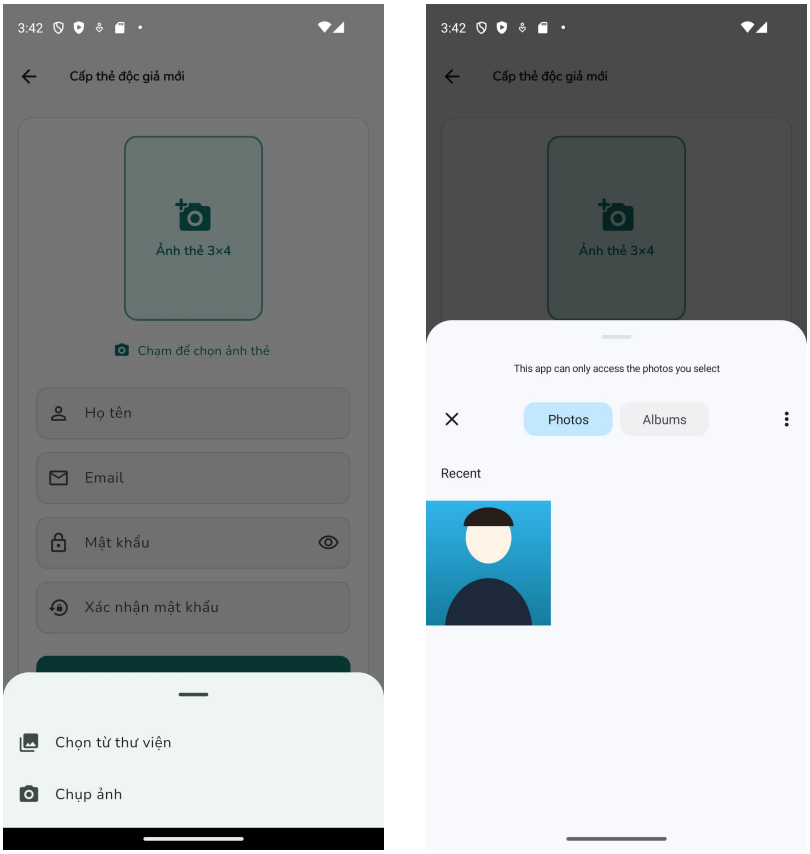



*Hình 19. Đăng nhập thủ thư và kiểm tra dữ liệu nhập trên Android*

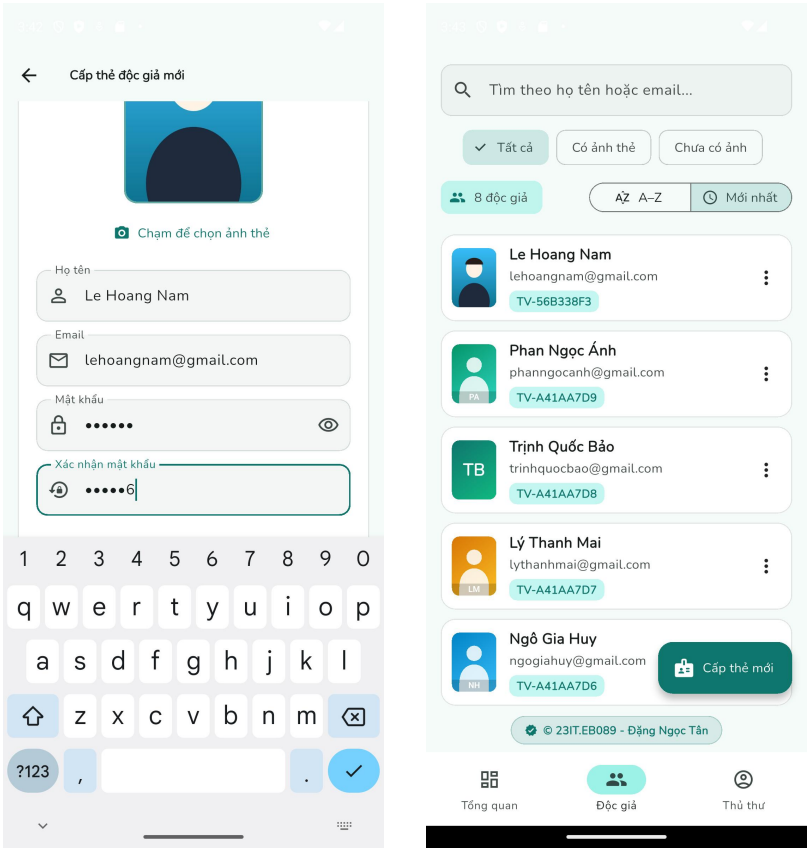



*Hình 20. Trang Tổng quan và danh sách độc giả trên Android*

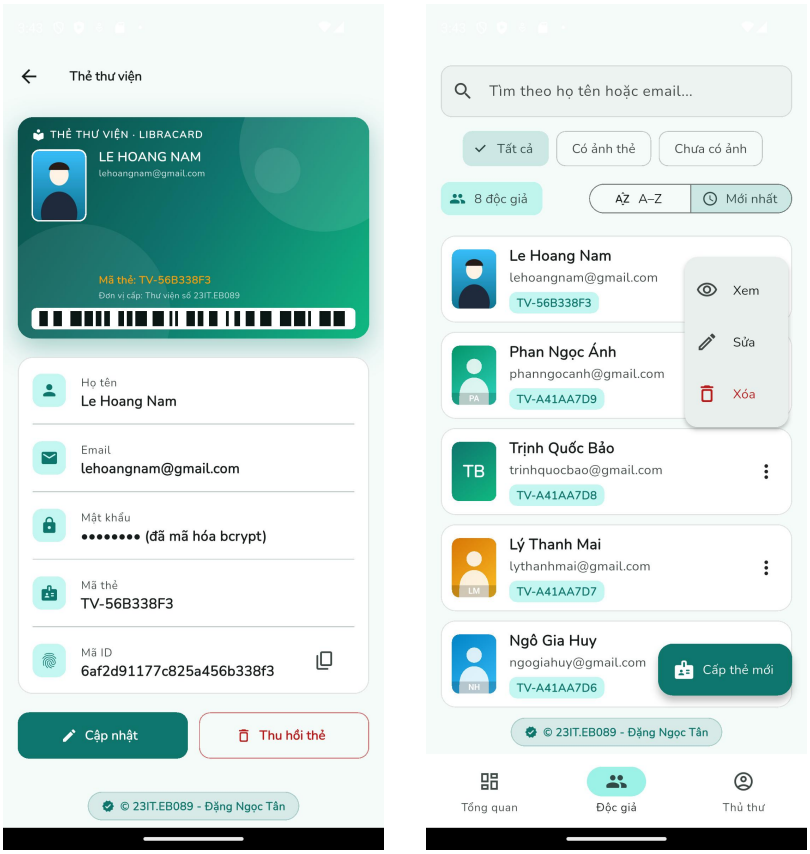
**Nhập hình ảnh trên di động (Câu 3):** chạm vào khung "Ảnh thẻ 3×4" để mở bảng chọn nguồn ảnh, gồm "Chọn từ thư viện" và "Chụp ảnh" bằng camera. Trình chọn ảnh hệ thống của Android hiện ra, chọn ảnh xong thì ảnh hiển thị ngay trong khung.



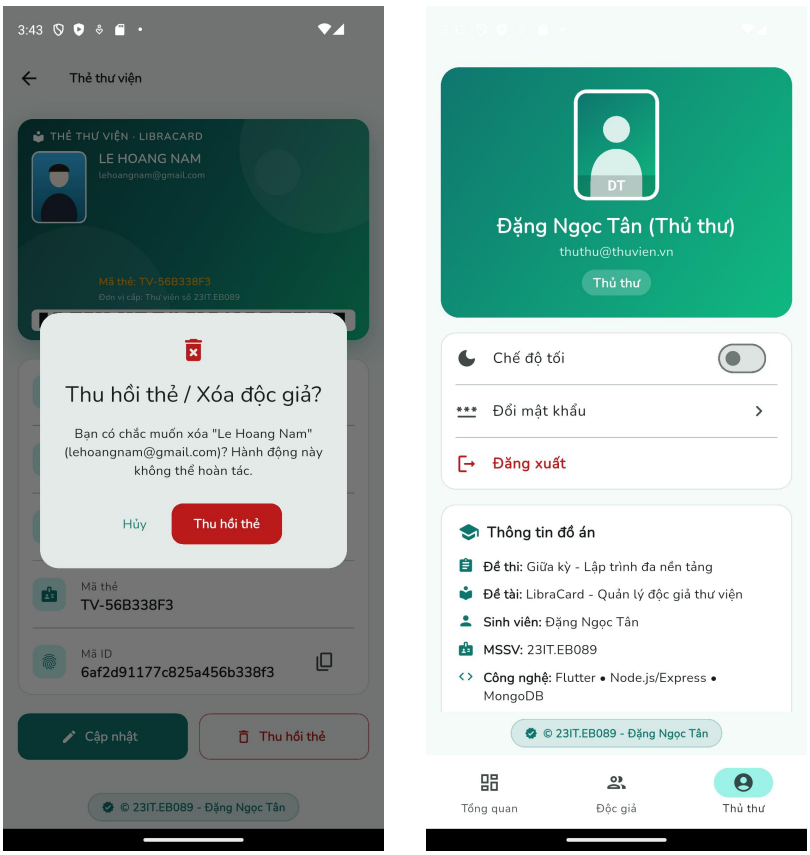

*Hình 21. Chọn nguồn ảnh (thư viện/camera) và trình chọn ảnh của Android*




*Hình 22. Nhập thông tin kèm ảnh thẻ; cấp thẻ thành công (sắp xếp Mới nhất)*




*Hình 23. Hiển thị thông tin độc giả trên thẻ thư viện và menu thao tác trên Android*




*Hình 24. Xác nhận thu hồi thẻ và trang Thủ thư trên Android*

## 6. Các chức năng nâng cao

- **Bảo mật:** mật khẩu mã hóa bcrypt; xác thực bằng JWT (hạn 7 ngày); API trả 401 khi token không hợp lệ và app tự đăng xuất.
- **Ghi nhớ đăng nhập:** token lưu bằng shared_preferences, mở lại app không cần đăng nhập lại.
- **Tìm kiếm và lọc:** tìm realtime theo họ tên hoặc email (debounce 400ms); lọc Tất cả / Có ảnh thẻ / Chưa có ảnh.
- **Sắp xếp kiểu tiếng Việt:** A–Z theo tên (từ cuối của họ tên) sau khi bỏ dấu, hoặc theo độc giả Mới nhất.
- **Thống kê:** tổng độc giả, số đã có hoặc chưa có ảnh thẻ, tỉ lệ hoàn thiện ảnh thẻ, 5 độc giả mới đăng ký.
- **Thẻ thư viện điện tử:** ảnh thẻ, mã thẻ tính từ _id, mã vạch vẽ bằng CustomPainter; form có ô xem trước thẻ cập nhật ngay khi nhập.
- **Kiểm tra dữ liệu:** định dạng email, mật khẩu tối thiểu 6 ký tự, xác nhận mật khẩu; báo lỗi khi email bị trùng (HTTP 409).
- **An toàn thao tác:** thu hồi thẻ có hộp thoại xác nhận, không cho thủ thư xóa chính mình; vuốt để xóa trên di động.

- **Giao diện:** Material 3 tông teal–emerald, font Nunito, responsive (Web: menu bên trái và bảng; Mobile: thanh điều hướng dưới và thẻ), hiệu ứng chuyển cảnh, Dark mode.
- **Sao lưu/khôi phục CSDL:** npm run export và npm run import với file database/docgia.json.

## 7. Hướng dẫn chạy và sản phẩm nộp

```
# 1) Khởi động MongoDB
start-mongodb.bat
# 2) Khởi động API
cd server && npm install && npm run import && npm start
# 3) Chạy app
cd app && flutter pub get
flutter run -d chrome            # Web
flutter run -d emulator-5554     # Android
```

**Tài khoản thủ thư demo:** thuthu@thuvien.vn / thuthu123. Các độc giả mẫu dùng mật khẩu 123456.

| Sản phẩm | Vị trí |
|---|---|
| Mã nguồn (GitHub) | https://github.com/ngoctan2503/GiuaKiDNT-23IT.EB089 |
| File cơ sở dữ liệu | database/docgia.json |
| Báo cáo | docs/BaoCao_23IT.EB089_DangNgocTan.docx (kèm bản .pdf) |
| Ảnh minh họa | docs/images/ (w*: Web, m*: Android, db*: CSDL) |
| Video demo | (dán link video demo tại đây) |

## 8. Kết luận

Ứng dụng LibraCard đáp ứng đủ các yêu cầu của đề. Thủ thư có thể cấp thẻ (thêm), xem (hiển thị), cập nhật (sửa) và thu hồi thẻ (xóa) độc giả trên nền tảng Web với dữ liệu lưu trong MongoDB. Ứng dụng có đăng nhập và nhiều chức năng nâng cao. Trên Android, thủ thư có thể nhập ảnh thẻ từ thư viện hoặc camera và xem thông tin độc giả trên thẻ thư viện. Cơ sở dữ liệu chỉ gồm 4 trường email, matkhau, hoten, hinhanh đúng như yêu cầu, và được nộp kèm dưới dạng file JSON.

Hướng phát triển: triển khai API và CSDL lên cloud (MongoDB Atlas, Render), in thẻ thư viện ra PDF, quét mã vạch bằng camera và build bản iOS.